Literature Survey: Chunking, Embedding and Retrieval Strategies for RAG

1. Introduction and Scope

Retrieval-Augmented Generation (RAG) grounds a language model in external documents by retrieving relevant text at query time. Three design decisions largely determine what the model sees: how documents are split into chunks, which embedding model represents those chunks, and how retrieval is performed and refined. This survey reviews work on each decision, identifies what is still unknown, and positions our project: a systematic comparison of chunking strategies and embedding models, extended with retrieval pipeline variants and efficiency measures, across more than one document type.

The survey is organised by theme: RAG foundations (Section 2), chunking (Section 3), embeddings (Section 4), retrieval pipelines (Section 5) and evaluation (Section 6). Section 7 compares the key works, Section 8 states the research gap, and Section 9 lists our hypotheses.

2. RAG Foundations

Lewis et al. [1] introduced RAG as a way to combine a parametric generator with a non-parametric retrieval memory for knowledge-intensive tasks. Dense retrieval with learned embeddings, popularised by Karpukhin et al. [2], made it practical to retrieve passages by semantic similarity instead of keyword overlap. Gao et al. [3] survey the field and group systems into naive, advanced and modular RAG, where advanced systems add pre-retrieval steps such as query rewriting and post-retrieval steps such as reranking.

Several studies show that retrieval is where RAG systems most often fail. Barnett et al. [4] report seven failure points from real engineering experience, several of which concern content that is missing, ranked too low or not usable in the final context. Liu et al. [5] show that language models use information less reliably when it sits in the middle of a long context, which argues for compact, precise retrieved context rather than simply retrieving more text. Cuconasu et al. [6] find that the kind and amount of irrelevant text in the retrieved context changes answer accuracy in non-obvious ways, so noise in retrieval needs to be measured rather than assumed harmless.

Takeaway: retrieval quality bounds answer quality, and both the amount and the precision of retrieved text matter. This motivates metrics that capture recall, precision and noise together.

3. Chunking Strategies

The default in most frameworks is fixed-size chunking with optional overlap, which is simple and fast but ignores document structure. Sentence-based splitting respects sentence boundaries. Semantic chunking places boundaries where the similarity between neighbouring sentences drops; it became popular through practitioner tutorials, but Qu et al. [7] ask whether its extra computation is worth it and report that its gains over fixed-size chunking are not consistent enough to justify the cost. This makes a controlled comparison across datasets, rather than a single benchmark, especially useful.

Other work changes the unit of retrieval. Chen et al. [8] study retrieval granularity and show that indexing propositions (small, self-contained facts) can improve retrieval and downstream question answering compared with passages or sentences, at the price of extra LLM preprocessing. Yepes et al. [9] propose element-based chunking for financial reports, using document structure such as titles and tables instead of fixed token counts, and report better retrieval and answer quality than fixed-size baselines. Finardi et al. [10] examine how the retriever, the chunking choice and the generator jointly affect RAG quality, showing that these components should be studied together rather than in isolation.

A second line of work enriches chunks with context. Late chunking [11] embeds a long text first and then pools token embeddings into chunk vectors, so each chunk keeps information from its surroundings. Contextual Retrieval [12] prepends a short, LLM-written description of the chunk's place in its document before embedding. Parent-document (small-to-big) retrieval, available in common frameworks, embeds small chunks for precise matching but returns the larger parent section to the generator.

Takeaway: the best chunking method seems to depend on document structure and task. Structure-aware and hierarchical methods are promising but have mostly been tested on a single domain, while the benefit of semantic chunking is disputed.

4. Embedding Models

Sentence-BERT [13] established bi-encoder sentence embeddings. Modern open models such as E5 [14], BGE [15], GTE [16] and Nomic Embed [17] are trained with large-scale contrastive learning and are competitive on public leaderboards. The standard benchmarks, MTEB [18] and BEIR [19], measure performance averaged across many tasks and largely evaluate retrieval over passages that are already segmented. They therefore say little about how an embedding model behaves when the corpus must first be chunked, or how its rank changes between document types such as code documentation and legal text.

Embedding size is also a practical concern. Matryoshka Representation Learning [20] trains vectors so that their leading dimensions remain useful when truncated. Binary and scalar quantization can shrink indexes and speed up search while keeping much of the retrieval quality [21]. These results come from benchmark settings, and their interaction with chunk size and document type has not been studied systematically.

Takeaway: leaderboard rankings may not transfer to a given corpus, and efficiency techniques such as truncation and quantization need to be evaluated together with the chunking choice.

5. Retrieval Pipelines

Sparse retrieval with BM25 [22] remains a strong baseline, especially for exact terms, identifiers and code tokens that embeddings can blur. Hybrid retrieval combines sparse and dense rankings; Reciprocal Rank Fusion [23] is a simple, training-free way to merge them. Cross-encoder rerankers [24] re-score a small candidate set jointly with the query and usually improve ordering at the cost of latency. Wang et al. [25] evaluate combinations of RAG components, including hybrid search and reranking, and recommend configurations that balance quality and efficiency. FAISS [26] is the standard library for efficient similarity search over dense vectors.

Takeaway: a stronger retrieval pipeline may reduce how much the chunking choice matters, but this has not been tested directly across chunking strategies. We test it as hypothesis H2.

6. Evaluation of RAG Systems

RAGAS [27] defines reference-free metrics for RAG, including context precision, context recall and faithfulness, many of which rely on an LLM judge. Zheng et al. [28] show that strong LLM judges agree well with humans on many tasks but exhibit position, verbosity and self-enhancement biases. Chunk-based retrieval metrics are also awkward to compare across chunkers, because a larger chunk may count as a hit simply because it contains more text.

For retrieval we therefore plan to locate gold evidence as character spans inside the source documents and score every method by span overlap, which is independent of chunk size and needs no LLM. LLM judging is kept for answer-level groundedness and correctness, on a smaller subset, with human verification of the question set and a cross-provider check of the judge.

Work	Focus	Main finding	Relation to our study
Lewis et al. [1]	RAG foundations	Combines a parametric generator with a non-parametric retrieval memory	Basis of our RAG pipeline
Gao et al. [3]	RAG survey	Groups systems into naive, advanced and modular RAG	Frames our pipeline variants: dense, hybrid, rerank
Barnett et al. [4]	RAG failure points	Seven failure points; several concern missing, low-ranked or unusable retrieved content	Motivates retrieval-quality metrics
Qu et al. [7]	Semantic vs fixed-size chunking	Semantic chunking gains are not consistent enough to justify its cost	Chunking is a main variable; we add document types, embedders and reranking
Chen et al. [8]	Retrieval granularity	Proposition units can improve retrieval and QA, but need LLM preprocessing	We cover cheaper strategies and add parent-child retrieval
Yepes et al. [9]	Element-based chunking for financial reports	Structure-aware chunking improves retrieval and QA	Single domain; we test structure-aware chunking on two document types
Finardi et al. [10]	Retriever, chunk and generator choices	Components interact and should be studied together	Supports our factorial, staged design
Günther et al. [11]	Late chunking	Embeds long text first, then pools token embeddings into chunk vectors	Contextual chunk enrichment; we compare against standard chunking
Anthropic [12]	Contextual Retrieval	Prepends an LLM-written description of the chunk's document context before embedding	Contextual enrichment for structured and legal documents
Reimers and Gurevych [13]	Sentence-BERT	Establishes bi-encoder sentence embeddings	Baseline embedding architecture
Wang et al. [14]	E5	Weakly supervised contrastive pre-training; strong open embedding model	One of our open embedding candidates

Xiao et al. [15]	BGE	Open general-purpose embedding family	One of our open embedding candidates
Kusupati et al. [20]	Matryoshka Representation Learning	Leading dimensions remain useful when vectors are truncated	Supports H3: truncated embeddings
Shakir et al. [21]	Binary and scalar quantization	Quantization shrinks indexes and speeds search while keeping much retrieval quality	Supports H3: storage and latency vs accuracy
Robertson and Zaragoza [22]	BM25	Strong sparse baseline for exact terms, identifiers and code tokens	Sparse half of our hybrid retrieval mode
Cormack et al. [23]	Reciprocal Rank Fusion	Simple, training-free way to merge sparse and dense rankings	Fusion method for our hybrid mode
Nogueira and Cho [24]	Cross-encoder reranking	Re-scores a candidate set jointly with the query; improves ordering at extra latency	Reranking mode used to test H2
Wang et al. [25]	RAG best practices	Recommends balanced combinations of RAG components, including hybrid search and reranking	General-purpose; we break results down by document type
Es et al. [27]	RAGAS	Reference-free RAG metrics including context precision, context recall and faithfulness	Span-based retrieval metrics; RAGAS-style answer-level checks
Zheng et al. [28]	LLM-as-a-judge	Strong LLM judges agree with humans but show position, verbosity and self-enhancement biases	Motivates cross-provider judge check and human verification

8. Research Gap

Based on the works we reviewed, the following gaps remain:

1. **Limited document diversity.** Chunking and embedding comparisons usually rely on one generic or single-domain corpus, so it is unclear whether a recommendation transfers between, for example, API documentation and legal text.
2. **Components studied in isolation.** Chunking, embedding model and retrieval pipeline (hybrid search, reranking) are rarely varied together, so their interactions are unknown. In particular, it is untested whether reranking reduces the importance of the chunking choice.
3. **Metrics tied to chunk boundaries.** Common retrieval metrics depend on how a corpus was chunked, which complicates fair comparison between chunkers of different sizes.
4. **Efficiency studied separately.** Embedding truncation and quantization are evaluated on benchmarks, not jointly with chunking, latency and index size.
5. **No practitioner-ready output.** Findings are seldom turned into a decision aid that maps document type and priorities (accuracy, speed, cost) to a recommended configuration.

9. Proposed Contribution and Hypotheses

Our project addresses these gaps with a staged empirical study of nine chunking strategies (fixed-size, sentence-based, semantic, structure-aware and parent-child), several open and optional API embedding models, and three retrieval modes (dense, hybrid, hybrid with reranking), evaluated on two document types with span-based metrics, latency and index size. The results feed a decision matrix and a small advisor application.

Research questions. (RQ1) Does the best chunking and embedding combination depend on the document type? (RQ2) Does a stronger retrieval pipeline reduce the sensitivity to chunking? (RQ3) How much accuracy is lost when embeddings are truncated or quantized?

Hypotheses.

- **H1:** Structure-aware chunking outperforms fixed-size and semantic chunking on structured documents such as API documentation and legal text.
- **H2:** A reranker narrows the performance gap between chunking strategies.
- **H3:** Truncated or binary embeddings lose little accuracy while greatly reducing storage and latency.

References

[1] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. NeurIPS, 2020.

[2] V. Karpukhin et al., "Dense passage retrieval for open-domain question answering," in Proc. EMNLP, 2020.

[3] Y. Gao et al., "Retrieval-augmented generation for large language models: A survey," arXiv:2312.10997, 2023.

[4] S. Barnett et al., "Seven failure points when engineering a retrieval augmented generation system," in Proc. IEEE/ACM CAIN, 2024, arXiv:2401.05856.

[5] N. F. Liu et al., "Lost in the middle: How language models use long contexts," Trans. Assoc. Comput. Linguist., vol. 12, 2024.

[6] F. Cuconasu et al., "The power of noise: Redefining retrieval for RAG systems," in Proc. ACM SIGIR, 2024.

[7] R. Qu, R. Tu, and F. Bao, "Is semantic chunking worth the computational cost?" in Findings of NAACL, 2025, arXiv:2410.13070.

[8] T. Chen et al., "Dense X retrieval: What retrieval granularity should we use?" in Proc. EMNLP, 2024, arXiv:2312.06648.

[9] A. J. Yepes et al., "Financial report chunking for effective retrieval augmented generation," arXiv:2402.05131, 2024.

[10] P. Finardi et al., "The chronicles of RAG: The retriever, the chunk and the generator," arXiv:2401.07883, 2024.

[11] M. Günther et al., "Late chunking: Contextual chunk embeddings using long-context embedding models," arXiv:2409.04701, 2024.

[12] Anthropic, "Introducing Contextual Retrieval," Anthropic blog, Sep. 2024.

[13] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. EMNLP-IJCNLP, 2019.

[14] L. Wang et al., "Text embeddings by weakly-supervised contrastive pre-training," arXiv:2212.03533, 2022.

[15] S. Xiao et al., "C-Pack: Packed resources for general Chinese embeddings," in Proc. ACM SIGIR, 2024, arXiv:2309.07597.

[16] Z. Li et al., "Towards general text embeddings with multi-stage contrastive learning," arXiv:2308.03281, 2023.

[17] Z. Nussbaum et al., "Nomic Embed: Training a reproducible long context text embedder," arXiv:2402.01613, 2024.

[18] N. Muennighoff et al., "MTEB: Massive text embedding benchmark," in Proc. EACL, 2023.

[19] N. Thakur et al., "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," in Proc. NeurIPS Datasets and Benchmarks, 2021.

[20] A. Kusupati et al., "Matryoshka representation learning," in Proc. NeurIPS, 2022.

[21] A. Shakir, T. Aarsen, and S. Lee, "Binary and scalar embedding quantization for significantly faster and cheaper retrieval," Hugging Face blog, Mar. 2024.

[22] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," Found. Trends Inf. Retr., vol. 3, no. 4, 2009.

[23] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in Proc. ACM SIGIR, 2009.

[24] R. Nogueira and K. Cho, "Passage re-ranking with BERT," arXiv:1901.04085, 2019.

[25] X. Wang et al., "Searching for best practices in retrieval-augmented generation," in Proc. EMNLP, 2024.

[26] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," IEEE Trans. Big Data, vol. 7, no. 3, 2021.

[27] S. Es et al., "RAGAs: Automated evaluation of retrieval augmented generation," in Proc. EACL (Demos), 2024.

[28] L. Zheng et al., "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in Proc. NeurIPS Datasets and Benchmarks, 2023.